

Gated Recurrent Unit (GRU)

1. Intro: Why GRU?

After LSTMs became popular, researchers realized:

- LSTMs are **very effective** at handling long-term dependencies.
 - But they are also **complex** — having **three gates** (input, forget, output) and a **cell state**.
 - This complexity means **more parameters, slower training**, and sometimes overkill for simpler tasks.
-
- So in 2014, **Cho et al.** proposed the **Gated Recurrent Unit (GRU)** — a **simpler version** of LSTM that:
 - Keeps the benefits of remembering long-term information
 - Reduces the number of gates and parameters
 - Trains faster and still performs very well
-

2. What is GRU?

GRU is a type of **Recurrent Neural Network (RNN)** designed to handle **sequential data** (like text, time series, speech, etc.) more effectively than basic RNNs.

The key idea:

GRU **decides how much past information to keep** and **how much new information to add** using two gates:

- **Update Gate (z_t)**
- **Reset Gate (r_t)**

Unlike LSTM, GRU **does not have a separate cell state** — it combines the cell state and hidden state into one vector (h_t).

3. The Idea Behind GRU

Think of GRU as a **smart memory manager**.

At every time step (each new word or data point), GRU must decide:

- How much of the **past** should I remember?
- How much of the **new input** should I keep?

The **update gate** helps remember important past information.

The **reset gate** helps forget irrelevant past information.

Together, they balance memory and learning dynamically.

4. The Setup

For each time step t :

- Input: (x_t) (current input, e.g., current word)
- Previous hidden state: (h_{t-1})
- Output: (h_t) (current hidden state)

The GRU uses two gates to compute (h_t):

1 Update Gate (z_t):

Decides **how much of the past information** to carry forward.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

If z_t is close to 1 $\rightarrow$ mostly keep the past

If z_t is close to 0 $\rightarrow$ mostly take new input

2 Reset Gate (r_t):

Decides **how much of the past** to forget when computing the new memory.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

If r_t is 0 $\rightarrow$ ignore past state (reset memory)

If r_t is 1 $\rightarrow$ keep past state fully

5. The Internal Operations of GRU

Once the two gates are computed, the GRU calculates:

❖ Candidate Hidden State ($\tilde{h}_t$):

This represents the new memory, combining input and partially reset past information.

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

Here, the **reset gate** controls how much of the past influences this step.

❖ Final Hidden State ((h_t)):

Finally, GRU decides how to combine the **old memory** (h_{t-1}) and **new memory** ($\tilde{h}_t$) using the **update gate**.

$$h_t = (1 - z_t) * \tilde{h}_t + z_t * h_{t-1}$$

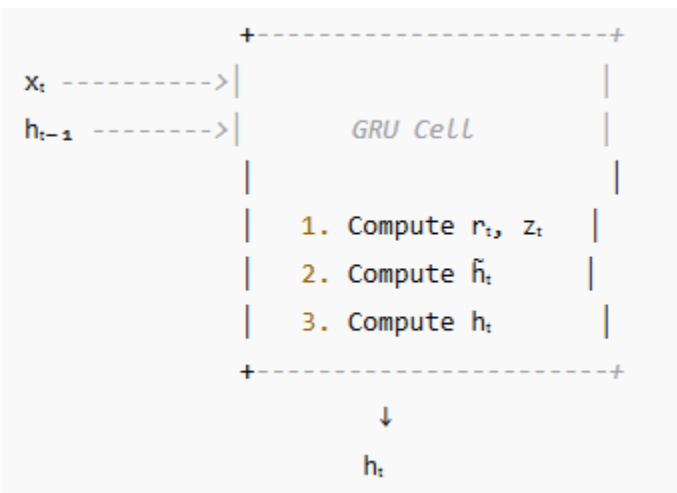
In words:

- If $z_t = 0 \rightarrow$ use mostly new memory ($\tilde{h}_t$)
- If $z_t = 1 \rightarrow$ keep old memory (h_{t-1})

This equation is the **heart** of GRU.

6. The Architecture (Step-by-Step)

Here's the flow of information inside one GRU cell:



You can think of GRU as “LSTM without the extra baggage” — simpler, yet powerful.

7. Understanding the Hidden State ((h_t))

In GRU, **hidden state = memory**.

This is a major difference from LSTM, where you had:

- Cell state ((C_t)): long-term memory
- Hidden state ((h_t)): short-term output memory

GRU merges both into **one state**, so:

h_t contains both short-term and long-term info.

This makes computation simpler and faster.

8. The Reset Gate in Action

When the **reset gate** (r_t) is near **0**, the GRU forgets the previous hidden state completely.

This is useful when the new input is **unrelated to the past**.

Example:

- Sequence: “I went to the restaurant. The food was delicious.”
- When the model starts processing the next sentence, like “Next day, I drove to work,”
the reset gate **resets** memory from the old topic (restaurant).

This helps GRU handle *context switches* effectively.

9. Mathematical Summary

Here’s the full GRU math compactly:

$$\begin{aligned}z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\\tilde{h}_t &= \tanh(W \cdot [r_t * h_{t-1}, x_t]) \\h_t &= (1 - z_t) * \tilde{h}_t + z_t * h_{t-1}\end{aligned}$$

That’s it — just 4 equations compared to LSTM’s 7–8!

This simplicity is what makes GRU fast and elegant.

10. LSTM vs GRU — Key Differences

Feature	LSTM	GRU
Number of Gates	3 (input, forget, output)	2 (update, reset)
Cell State	Has separate cell state (C_t)	Merged with hidden state (h_t)
Complexity	More parameters	Fewer parameters
Training Speed	Slower	Faster
Performance	Great for long dependencies	Similar or better in small datasets
Memory Usage	Higher	Lower
Gate Control	More fine-tuned	Simpler but effective

In summary:

- **LSTM** = more precise memory control
- **GRU** = simpler, faster, nearly equal performance

11. Advantages of GRU

- Simpler than LSTM (fewer parameters)
- Faster to train
- Works well on smaller datasets
- Handles vanishing gradients better than RNN

- Performs comparably to LSTM in many NLP and speech tasks

12. Limitations

- May not capture very long-term dependencies as precisely as LSTM
- Slightly less flexible (since it merges cell and hidden state)
- Some tasks benefit from the extra control that LSTMs provide

13. GRU in Keras (Code Example)

A typical GRU model in Keras looks like this:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense, Embedding

model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=128, input_length=seq_len),
    GRU(128, return_sequences=False),
    Dense(vocab_size, activation='softmax')
])

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

This setup works for **text generation**, **next-word prediction**, or **sentiment analysis**.

14. Summary (Simple Recap)

Concept	Description
GRU Goal	Handle sequential data efficiently
Key Idea	Simplified LSTM with 2 gates
Gates	Update (z_t) and Reset (r_t)
Memory	Uses only hidden state (h_t)
Benefit	Faster, fewer parameters
Key Equation	$\tilde{h}_t = (1 - z_t) \tilde{h}_t + z_t h_{t-1}$
Best For	Time series, NLP, speech tasks

In Short:

The **Gated Recurrent Unit (GRU)** is a streamlined version of LSTM — faster, simpler, and just as smart in remembering what matters and forgetting what doesn't.
